LangChain, LlamaIndex & Haystack: A Complete Guide

Nov 3, 2026 · @V

1. Introduction

LangChain, LlamaIndex and Haystack are the three most widely used open-source Python frameworks for building applications on top of large language models (LLMs). All three solve the same basic problem: an LLM on its own knows nothing about your data, cannot take actions, and forgets everything between calls. These frameworks supply the plumbing that connects a model to documents, databases, tools and memory.

They overlap heavily, but each grew from a different starting point:

- **LangChain** began as a toolkit for *chaining* LLM calls and tools together, and has grown into a broad ecosystem centred on agents (via LangGraph) and observability (via LangSmith).
- **LlamaIndex** began as a way to *index* private data for LLMs, and remains the most data- and retrieval-focused of the three.
- **Haystack**, built by the Berlin company deepset, began as a search and question-answering framework and is designed around explicit, production-grade *pipelines*.

What these frameworks help you build

Application type	What it does	Typical example
Retrieval-Augmented Generation (RAG)	Fetches relevant passages from your documents and gives them to the LLM as context	Chat with company policies or product manuals
Agents	Lets the LLM decide which tools to call, in what order, until a task is done	A research assistant that searches the web and writes a report
Chatbots with memory	Keeps conversation history and user context across turns	Customer support assistant

Application type	What it does	Typical example
Data extraction	Turns unstructured text into structured records	Pulling invoice fields out of PDFs
Semantic search	Finds documents by meaning, not just keywords	Internal knowledge-base search

How to read this guide

Section 2 explains the concepts all three frameworks share, so the later chapters can focus on what is different. Sections 3 to 5 cover each framework in depth: architecture, core components, a working code example, and strengths and weaknesses. Section 6 compares them side by side, and Section 7 helps you choose. Version details reflect each project's state as of November 2026; these libraries change quickly, so check the official docs before you pin a version.

2. Core concepts shared by all three

All three frameworks are built from the same handful of building blocks; once you understand these, each framework is mostly a different way of wiring them together.

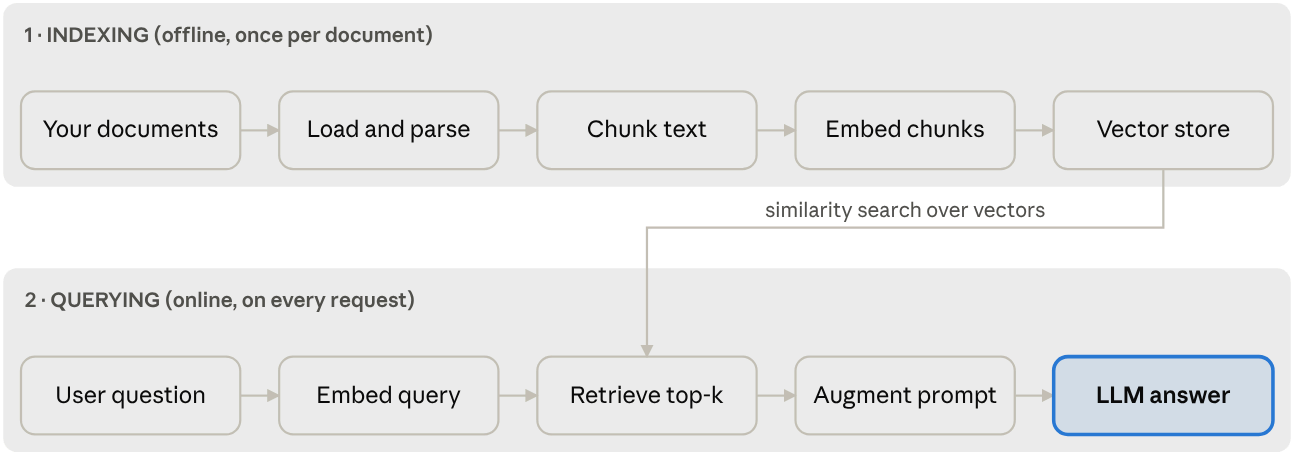
Concept	What it is	Why it matters
LLM / chat model	A model such as GPT, Claude, Gemini, Llama or Mistral, called through an API or run locally	The reasoning and text-generation engine
Prompt template	A reusable prompt with placeholders filled at run time	Keeps prompts consistent and testable
Document	A unit of text plus metadata (source, page, author)	The common currency between loaders, splitters and stores
Chunking / splitting	Cutting long documents into passages of a few hundred tokens	Models have limited context; small chunks retrieve more precisely
Embedding	A vector of numbers that captures the meaning of a text	Lets you find text by meaning instead of exact keywords

Concept	What it is	Why it matters
Vector store	A database that stores embeddings and finds the nearest ones (Pinecone, Qdrant, Weaviate, Chroma, pgvector, Elasticsearch, FAISS)	The searchable memory of a RAG system
Retriever	A component that takes a query and returns relevant chunks	The bridge between the question and your data
Re-ranker	A second model that re-orders retrieved chunks by relevance	Improves precision before the LLM sees the context
Tool	A function the LLM can call (search, calculator, SQL, an API)	Lets the model act, not just talk
Agent	A loop where the LLM picks a tool, observes the result and decides the next step	Handles open-ended, multi-step tasks
Memory	Stored conversation history or user facts	Makes multi-turn chat coherent

Retrieval-Augmented Generation (RAG)

RAG is the single most common pattern these frameworks are used for. Instead of retraining a model on your data, you search your data at question time and paste the most relevant passages into the prompt.

RAG grounds every answer in retrieved chunks of your own data



Retrieval-Augmented Generation · indexing and querying phases

Indexing runs once per document and fills the vector store; querying runs on every request, pulling the nearest chunks from that store into the prompt so the LLM answers from your data rather than from memory alone.

Agents and tool calling

An agent wraps the LLM in a loop. On each turn the model reads the goal and the results so far, then either calls a tool or returns a final answer. Modern chat models support *native tool calling*: you describe each tool with a name, a description and a JSON schema, and the model returns a structured request to call it. The framework executes the call, appends the result to the conversation and asks the model again.

The main design question is how much control you keep. A fully autonomous loop is flexible but harder to predict; an explicit graph or pipeline, where you define which steps can follow which, is easier to test and debug. Each framework answers this differently: LangChain through LangGraph's state graphs, LlamaIndex through event-driven Workflows, and Haystack through pipelines that may contain loops.

Evaluation and observability

LLM output is non-deterministic, so production systems need tracing (what was retrieved, what prompt was sent, how many tokens and how long each step took) and evaluation (is the answer faithful to the context, is the retrieved context relevant). Common tools include LangSmith, Langfuse, Arize Phoenix, Ragas and DeepEval; all three frameworks integrate with several of them.

3. LangChain

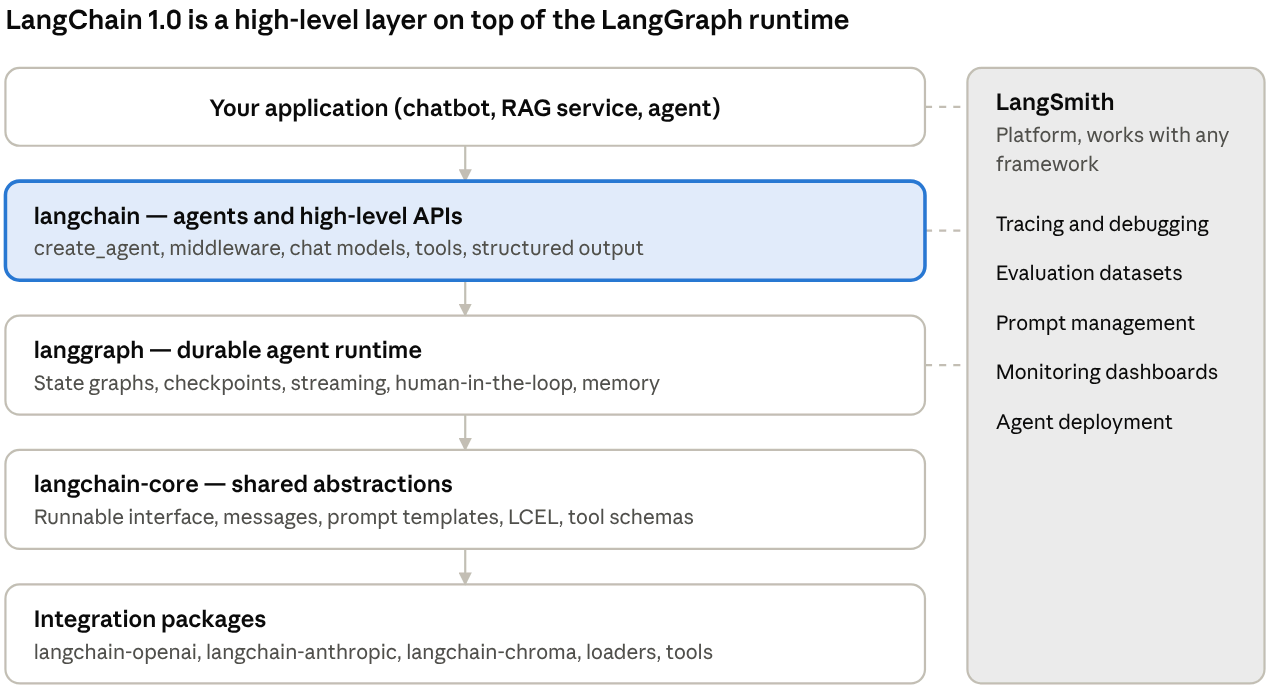
LangChain is the broadest and most widely adopted of the three frameworks, and since its 1.0 release it is best understood as a high-level agent API running on the LangGraph runtime. Its strength is breadth: hundreds of integrations, two first-class languages (Python and JavaScript/TypeScript) and a commercial platform, LangSmith, for tracing, evaluation and deployment.

3.1 History and evolution

Period	Milestone
Oct 2022	Harrison Chase releases LangChain as an open-source Python library for chaining LLM calls
2023	Rapid growth; LangChain, Inc. is founded; LangChain Expression Language (LCEL) and LangSmith are introduced
Early 2024	LangGraph launches as a low-level library for stateful, cyclic agent workflows
2024	The monolith is split into `langchain-core`, `langchain` and per-provider integration packages
Sep 2025	Alpha releases of LangChain 1.0 and LangGraph 1.0 (announcement)
Oct 2025	LangChain 1.0 and LangGraph 1.0 reach general availability (announcement)

With 1.0, the LangChain team promised no breaking changes until version 2.0 (changelog). The package was slimmed down to focus on agents, and older utilities such as legacy chains moved to a separate `langchain-classic` package.

3.2 Architecture



LangChain ecosystem · four layers plus LangSmith

Read it top down: your app calls `langchain`, which runs on `langgraph`, which builds on the shared types in `langchain-core`; provider packages plug in at the bottom, and LangSmith observes every layer from the side.

A common misreading is that LangGraph is an add-on to LangChain. Since 1.0 it is the reverse: LangGraph is the low-level runtime and LangChain is a high-level API built on it.

3.3 Core components

Component	Purpose	Key classes / functions
Chat models	Uniform interface to any LLM provider	`init_chat_model`, `ChatOpenAI`, `ChatAnthropic`
Messages and content blocks	Provider-neutral message format, including tool calls, images and reasoning	`HumanMessage`, `AIMessage`, `ToolMessage`
Prompt templates	Reusable, parameterised prompts	`ChatPromptTemplate`

Component	Purpose	Key classes / functions
Tools	Python functions the model can call	`@tool` decorator
Agents	The model-plus-tools loop	`create_agent`
Middleware	Hooks that run before or after model and tool calls (summarisation, human approval, PII redaction, retries)	`AgentMiddleware` subclasses
Structured output	Forces the answer into a Pydantic schema	`response_format`, `ToolStrategy`
Document loaders	Read PDFs, web pages, Notion, Google Drive and more	`PyPDFLoader`, `WebBaseLoader`
Text splitters	Chunk documents	`RecursiveCharacterTextSplitter`
Embeddings and vector stores	Index and search chunks	`OpenAIEmbeddings`, `Chroma`, `PGVector`
Retrievers	Return relevant documents for a query	`vectorstore.as_retriever()`
LCEL	Pipe syntax (`prompt | model | parser`) for composing Runnables	`Runnable`, `RunnableParallel`

3.4 LangGraph in brief

LangGraph models an application as a **state graph**. You define a typed state object, *nodes* (functions that read and update the state) and *edges* (which node runs next, optionally chosen by a condition). Because edges can loop back, a graph can retry, reflect or hand off between agents. Its production features are what set it apart:

- **Checkpointing**: state is saved after each step, so a run can resume after a crash or pause.
- **Human-in-the-loop**: a graph can pause for approval and continue when a person responds.

- **Streaming**: tokens, intermediate steps and state updates stream to the client.
- **Memory**: short-term memory lives in the state; long-term memory uses persistent stores.

3.5 Code example: a RAG agent

```
# pip install langchain langchain-openai langchain-chroma langchain-community
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 1. Indexing
docs = PyPDFLoader("handbook.pdf").load()
chunks = RecursiveCharacterTextSplitter(chunk_size=800,
chunk_overlap=100).split_documents(docs)
store = Chroma.from_documents(chunks, OpenAIEmbeddings())

# 2. Expose retrieval as a tool
@tool
def search_handbook(query: str) -> str:
    """Search the employee handbook for relevant passages."""
    hits = store.similarity_search(query, k=4)
    return "\n\n".join(d.page_content for d in hits)

# 3. Build and run the agent
agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[search_handbook],
    system_prompt="Answer HR questions using the handbook. Cite the passage
you used.",
)
result = agent.invoke({"messages": [{"role": "user", "content": "How many
leave days do I get?"}]})
print(result["messages"][-1].content)
```

The agent decides when to call `search_handbook`; for a fixed retrieve-then-answer flow you can instead compose an LCEL chain or a two-node LangGraph graph.

3.6 Strengths and weaknesses

Strengths	Weaknesses
Largest ecosystem of integrations and community examples	History of frequent API changes; many online tutorials target pre-1.0 APIs
First-class Python and JavaScript/TypeScript	Abstractions can feel heavy for simple tasks
LangGraph gives fine control over complex, long-running agents	Two libraries (LangChain and LangGraph) to learn for advanced work
LangSmith offers mature tracing, evaluation and deployment	Best observability and hosting features are commercial
Stable API commitment since 1.0	RAG-specific tooling is less deep than LlamaIndex's

4. LlamaIndex

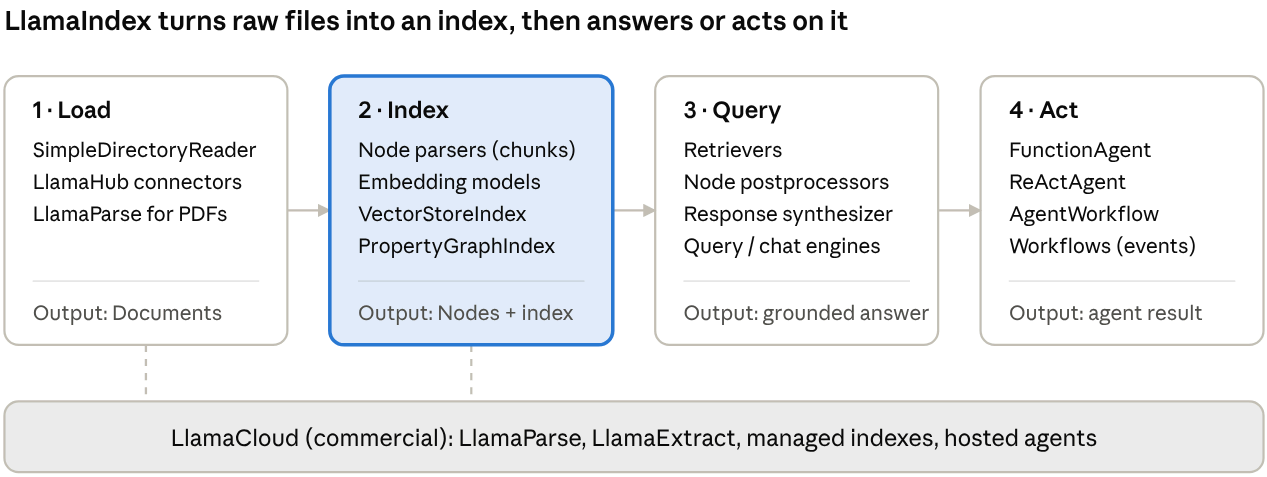
LlamaIndex is the data-first framework: it is strongest when your application revolves around reading, parsing and reasoning over documents. Its open-source core handles ingestion, indexing and querying, its Workflows engine handles agents, and its commercial LlamaCloud platform adds high-accuracy parsing of messy enterprise files.

4.1 History and evolution

Period	Milestone
Nov 2022	Jerry Liu releases "GPT Index", a way to index private data for GPT models
2023	Renamed LlamaIndex; LlamaIndex, Inc. is founded; LlamaHub connector library grows
Early 2024	v0.10 splits the project into `llama-index-core` plus hundreds of integration packages; LlamaParse launches
2024	Event-driven Workflows introduced as the foundation for agents

Period	Milestone
Jun 2025	Workflows 1.0 ships as a standalone package for Python and TypeScript (announcement)
2026	Push into document agents with hosted agents (LlamaAgents, in preview) and LiteParse, a local open-source parser

4.2 Architecture



LlamaIndex data pipeline · load, index, query, act

Data moves left to right: loaders produce Documents, the index stage turns them into embedded Nodes, query components retrieve and synthesise an answer, and agents or Workflows wrap the whole thing in tools and multi-step logic. LlamaCloud supplies managed parsing and indexing for the first two stages.

4.3 Core components

Component	Purpose	Key classes
Readers / connectors	Load files, databases, APIs and SaaS apps into `Document` objects	`SimpleDirectoryReader`, LlamaHub readers

Component	Purpose	Key classes
LlamaParse	Managed parser for complex PDFs, Word, PowerPoint and Excel, including tables and scanned pages	`LlamaParse`
Node parsers	Split Documents into Nodes (chunks) that keep links to their source and neighbours	`SentenceSplitter`, `SemanticSplitterNodeParser`, `HierarchicalNodeParser`
Ingestion pipeline	Repeatable chain of transformations with caching and de-duplication	`IngestionPipeline`
Indexes	Data structures over Nodes	`VectorStoreIndex`, `SummaryIndex`, `PropertyGraphIndex`, `KeywordTableIndex`
Storage	Pluggable vector, document and index stores	`StorageContext`
Retrievers	Fetch relevant Nodes, including hybrid, auto-merging and recursive retrieval	`VectorIndexRetriever`, `QueryFusionRetriever`
Node postprocessors	Re-rank, filter or trim retrieved Nodes	`SimilarityPostprocessor`, Cohere and LLM rerankers
Response synthesizers	Combine Nodes into an answer (`compact`, `refine`, `tree_summarize`)	`get_response_synthesizer`
Query and chat engines	Ready-made question-answering and conversational interfaces	`index.as_query_engine()`, `index.as_chat_engine()`
Agents	Tool-using agents and multi-agent hand-offs	`FunctionAgent`, `ReActAgent`, `AgentWorkflow`
Workflows	Event-driven, async orchestration of steps	`Workflow`, `@step`, `StartEvent`, `StopEvent`
Global settings	Default LLM, embedding model and chunk size	`Settings`

4.4 Workflows: the event-driven engine

Where LangGraph uses an explicit graph of nodes and edges, LlamaIndex Workflows use **events**. Each step is an async method decorated with `@step`; it receives one typed event and returns another. The runtime routes events to whichever step accepts that type, so branches are just `if` statements and loops are steps that emit an earlier event again. Workflows support shared typed state, streaming of intermediate events, human-in-the-loop pauses and OpenTelemetry-based observability. Since version 1.0 the engine is a separate package, `llama-index-workflows`, usable even outside LlamaIndex.

4.5 Code example: RAG in five lines, then an agent

```
# pip install llama-index
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader,
Settings
from llama_index.llms.openai import OpenAI

Settings.llm = OpenAI(model="gpt-4o-mini")

# 1. Load and index every file in ./data
documents = SimpleDirectoryReader("data").load_data()
index = VectorStoreIndex.from_documents(documents)

# 2. Ask a question
query_engine = index.as_query_engine(similarity_top_k=4)
print(query_engine.query("How many leave days do employees get?"))
```

The same index can become a tool for an agent:

```
import asyncio
from llama_index.core.agent.workflow import FunctionAgent
from llama_index.core.tools import QueryEngineTool

handbook_tool = QueryEngineTool.from_defaults(
    query_engine=query_engine,
    name="handbook",
    description="Answers questions about the employee handbook.",
)

agent = FunctionAgent(
```

```
    tools=[handbook_tool],
    llm=OpenAI(model="gpt-4o-mini"),
    system_prompt="You are an HR assistant. Use the handbook tool for policy
questions.",
)

async def main():
    response = await agent.run("Can I carry unused leave into next year?")
    print(response)

asyncio.run(main())
```

4.6 Strengths and weaknesses

Strengths	Weaknesses
Deepest RAG toolkit: many index types, advanced retrievers and synthesis modes	Agent tooling is capable but less widely adopted than LangGraph
Very fast start: a working RAG app in about five lines	High-level defaults can hide what is happening until you customise
LlamaParse handles tables, multi-column layouts and scanned PDFs well	Best parsing and managed indexing are paid LlamaCloud services
Clean event-driven Workflows, async by default	Large number of integration packages to track and version
Python and TypeScript implementations	Smaller observability story of its own; relies on third-party tools

5. Haystack

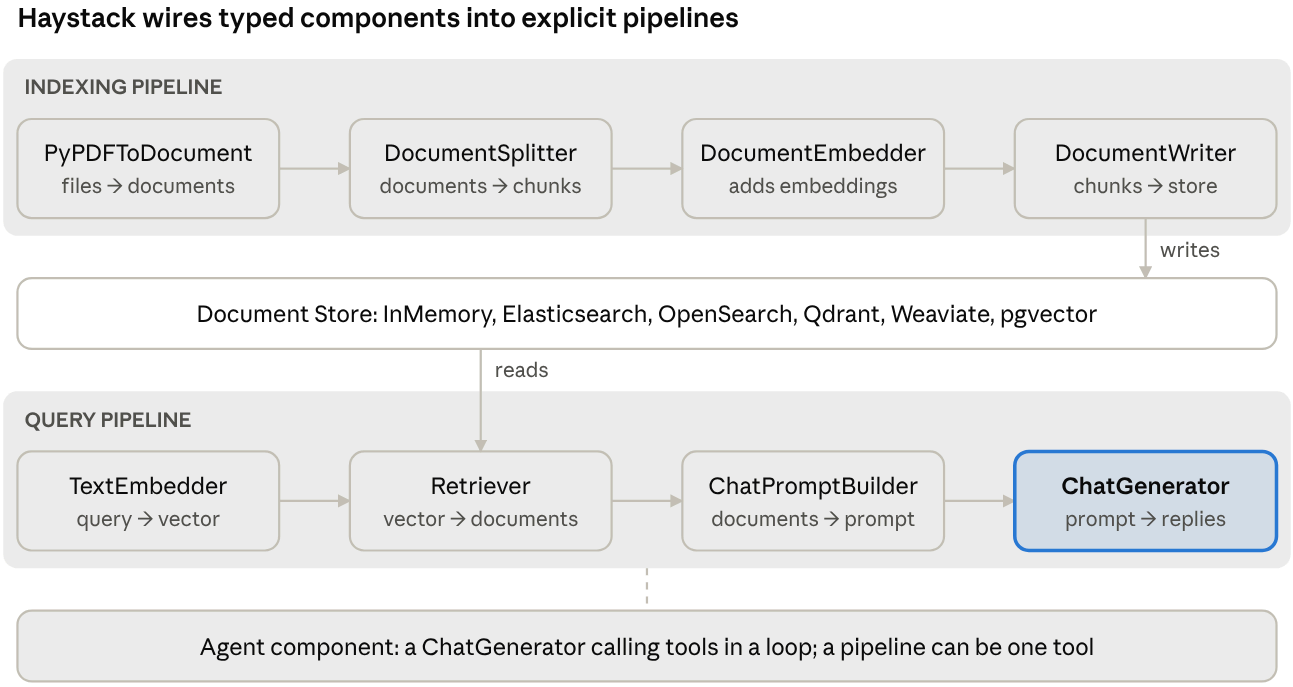
Haystack is the production-oriented framework: every application is an explicit pipeline of typed components, which makes systems easy to inspect, serialise to YAML, test and deploy. It is built by deepset, a Berlin company, is Apache 2.0 licensed, and is Python-only.

5.1 History and evolution

Period	Milestone
2019	deepset (founded by Milos Rusic, Malte Pietsch and Timo Möller) releases Haystack 1.x for extractive question answering and semantic search over documents (source)
2020–2023	1.x adds generative QA, LLM support and a large set of document stores
Mar 2024	Haystack 2.0, a complete rewrite, introduces typed components, explicit connections and pipelines that may contain loops
2025	A first-class `Agent` component, tool support and MCP integration arrive in 2.x releases
2026	The 2.x line continues with frequent minor releases; the commercial deepset platform is rebranded the Haystack Enterprise Platform (source)

If you find tutorials using `from haystack.nodes import ...` or `farm-haystack`, they are for the retired 1.x API. Current code installs `haystack-ai` and imports from `haystack.components`.

5.2 Architecture



Haystack 2.x · indexing and query pipelines sharing a Document Store

Two pipelines share one Document Store: the indexing pipeline converts, splits, embeds and writes documents; the query pipeline embeds the question, retrieves matching documents, builds a prompt and generates the reply. Each arrow is a named, type-checked connection between an output socket and an input socket.

5.3 Core concepts

- **Component**: a Python class decorated with `@component` that declares typed inputs and outputs and implements a `run()` method. Anything can be a component: a converter, a retriever, an LLM call, your own business logic.
- **Pipeline**: a directed graph of components. You `add_component()` each one and `connect()` an output to an input, for example `"retriever.documents"` to `"prompt_builder.documents"`. Haystack checks types at connect time, so mis-wiring fails early.
- **Branches and loops**: routers (`ConditionalRouter`, `FileTypeRouter`) send data down different branches, and loops let a pipeline retry or self-correct.
- **Document Store**: the storage layer, with integrations for in-memory, Elasticsearch, OpenSearch, Qdrant, Weaviate, Pinecone, Milvus, Chroma, pgvector and more.

- **Agent**: a ready-made component that runs a chat generator and a set of tools in a loop until an exit condition is met. Tools can be Python functions, MCP servers or whole pipelines.
- **Serialisation**: any pipeline can be saved to and loaded from YAML, so pipelines can be versioned and reviewed like configuration.
- **Secrets**: API keys are passed through a `Secret` class so they are never serialised into YAML by accident.

5.4 Key component families

Family	Examples	Role
Converters	`PyPDFToDocument`, `HTMLToDocument`, `DOCXToDocument`	Turn files into `Document` objects
Preprocessors	`DocumentCleaner`, `DocumentSplitter`	Clean and chunk text
Embedders	`SentenceTransformersDocumentEmbedder`, `OpenAITextEmbedder`	Create vectors for documents and queries
Writers	`DocumentWriter`	Store documents in a Document Store
Retrievers	`InMemoryBM25Retriever`, `InMemoryEmbeddingRetriever`, store-specific retrievers	Find relevant documents by keyword or vector
Rankers	`TransformersSimilarityRanker`, `LostInTheMiddleRanker`	Re-order results for relevance
Builders	`PromptBuilder`, `ChatPromptBuilder`	Fill Jinja2 prompt templates
Generators	`OpenAIChatGenerator`, `AnthropicChatGenerator`, `HuggingFaceAPIChatGenerator`, Ollama	Call the LLM
Routers and joiners	`ConditionalRouter`, `DocumentJoiner`	Branch and merge data flows, for example hybrid search
Agents and tools	`Agent`, `Tool`, `ToolInvoker`, MCP tools	Tool calling and agent loops

Family	Examples	Role
Evaluators	`FaithfulnessEvaluator`, `ContextRelevanceEvaluator`	Score RAG quality

5.5 Code example: a RAG pipeline

```
# pip install haystack-ai
from haystack import Pipeline, Document
from haystack.document_stores.in_memory import InMemoryDocumentStore
from haystack.components.retrievers.in_memory import InMemoryBM25Retriever
from haystack.components.builders import ChatPromptBuilder
from haystack.components.generators.chat import OpenAIChatGenerator
from haystack.dataclasses import ChatMessage

# 1. Store some documents
store = InMemoryDocumentStore()
store.write_documents([
    Document(content="Employees receive 24 days of paid leave per year."),
    Document(content="Up to 5 unused leave days can be carried into the next
year."),
])

# 2. Define the prompt
template = [ChatMessage.from_user(
    "Answer using only these documents:\n"
    "{% for d in documents %}- {{ d.content }}\n{% endfor %}\n"
    "Question: {{ question }}"
)]

# 3. Build the pipeline
pipe = Pipeline()
pipe.add_component("retriever", InMemoryBM25Retriever(document_store=store))
pipe.add_component("prompt_builder", ChatPromptBuilder(template=template))
pipe.add_component("llm", OpenAIChatGenerator(model="gpt-4o-mini"))
pipe.connect("retriever.documents", "prompt_builder.documents")
pipe.connect("prompt_builder.prompt", "llm.messages")

# 4. Run it
```

```
q = "Can I carry leave over?"
result = pipe.run({"retriever": {"query": q}, "prompt_builder": {"question":
q}})
print(result["llm"]["replies"][0].text)
```

An agent with a tool takes a few more lines:

```
from haystack.components.agents import Agent
from haystack.tools import tool

@tool
def leave_balance(employee_id: str) -> str:
    """Return the remaining leave days for an employee."""
    return "12 days"  # replace with a real HR API call

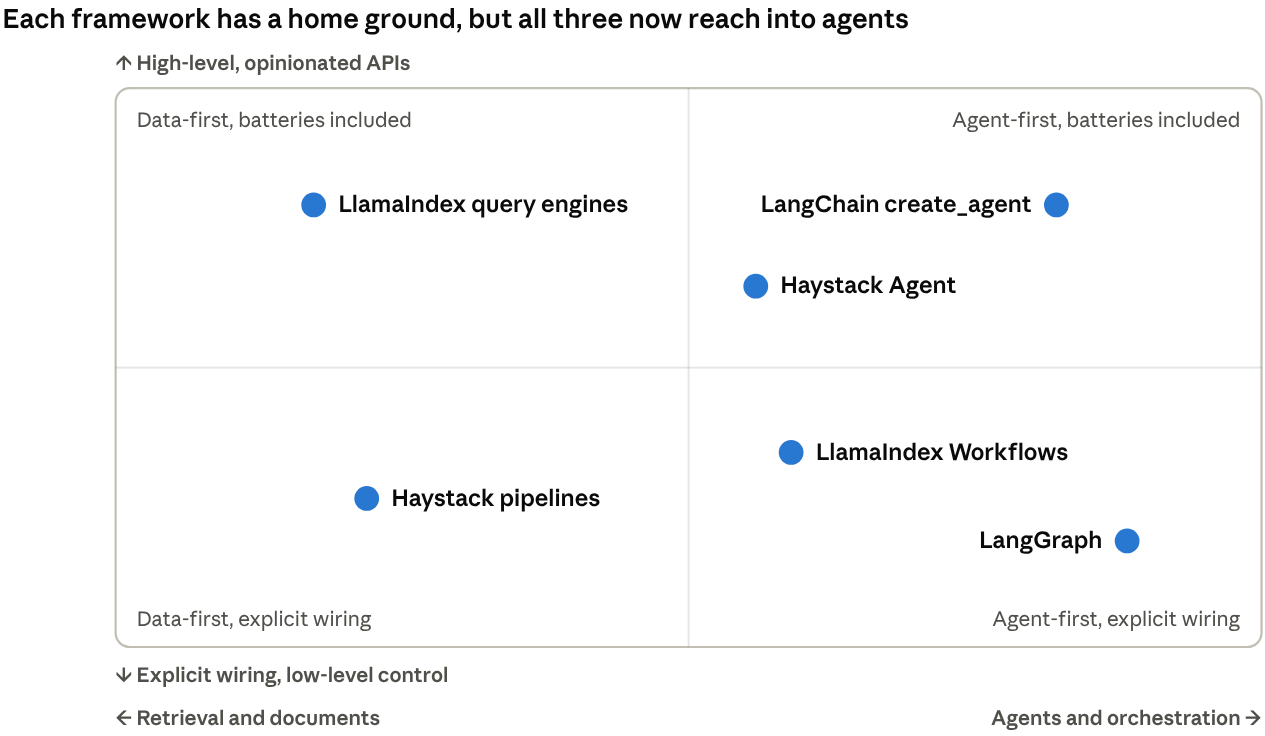
agent = Agent(
    chat_generator=OpenAIChatGenerator(model="gpt-4o-mini"),
    tools=[leave_balance],
    system_prompt="You are an HR assistant.",
)
result = agent.run(messages=[ChatMessage.from_user("How much leave does E-
1042 have?")])
print(result["messages"][-1].text)
```

5.6 Strengths and weaknesses

Strengths	Weaknesses
Explicit, type-checked pipelines are easy to read, debug and test	More code than LlamaIndex for a quick prototype
YAML serialisation suits teams that manage pipelines as configuration	Python only, no official JavaScript/TypeScript version
Strong heritage in search: BM25, hybrid retrieval, rankers, many document stores	Smaller community and fewer third-party tutorials than LangChain
Built-in evaluators and OpenTelemetry tracing	Agent ecosystem is younger than LangGraph's
Stable 2.x API with a clear deprecation policy	Enterprise platform features are commercial

6. Side-by-side comparison

In short: LangChain leads on agents and ecosystem breadth, LlamaIndex leads on document ingestion and retrieval, and Haystack leads on explicit, maintainable production pipelines. All three can build a solid RAG system and a tool-using agent; the difference is how much they help with each and how much control you keep.



Positioning map · focus versus level of abstraction (qualitative)

Positions are a qualitative judgement, not a measurement: the top half favours fast starts with sensible defaults, the bottom half favours explicit control you wire yourself.

6.1 Feature comparison

Dimension	LangChain (+ LangGraph)	LlamaIndex	Haystack
Maintainer	LangChain, Inc. (US)	LlamaIndex, Inc. (US)	deepset GmbH (Germany)
First release	2022	2022 (as GPT Index)	2019
Licence	MIT	MIT	Apache 2.0

Dimension	LangChain (+ LangGraph)	LlamaIndex	Haystack
Languages	Python, JavaScript/TypeScript	Python, TypeScript	Python
Core philosophy	Composable building blocks for agents	Connect LLMs to your data	Explicit, typed pipelines for production
Central abstraction	`create_agent`; LangGraph `StateGraph`	Index, query engine, Workflow	Component and `Pipeline`
Data ingestion	Many loaders and splitters	Richest: LlamaHub, LlamaParse, ingestion pipeline	Converters, cleaners, splitters
Retrieval depth	Good; standard retrievers plus integrations	Deepest: many index types, hybrid, auto-merging, recursive	Strong: BM25, dense, hybrid, rankers
Agents	Most mature: LangGraph graphs, middleware, multi-agent	Event-driven Workflows, `FunctionAgent`, `AgentWorkflow`	`Agent` component inside pipelines; loops and routers
Control flow model	Graph of nodes and edges with shared state	Typed events routed between async steps	Directed graph of typed components, loops allowed
Durable execution	Built in (checkpointers, resume, human-in-the-loop)	Workflow state, pause and resume	Pipeline breakpoints and snapshots
Observability	LangSmith (commercial), plus OpenTelemetry tools	Third-party: Arize Phoenix, Langfuse, OpenTelemetry	OpenTelemetry tracing, Langfuse and others
Evaluation	LangSmith evaluations, OpenEvals	Built-in evaluators, Ragas and others	Built-in evaluator components
Serialisation	Code-first	Code-first	YAML pipelines
Commercial platform	LangSmith (tracing, evals, deployment)	LlamaCloud (LlamaParse, LlamaExtract, indexes)	Haystack Enterprise Platform

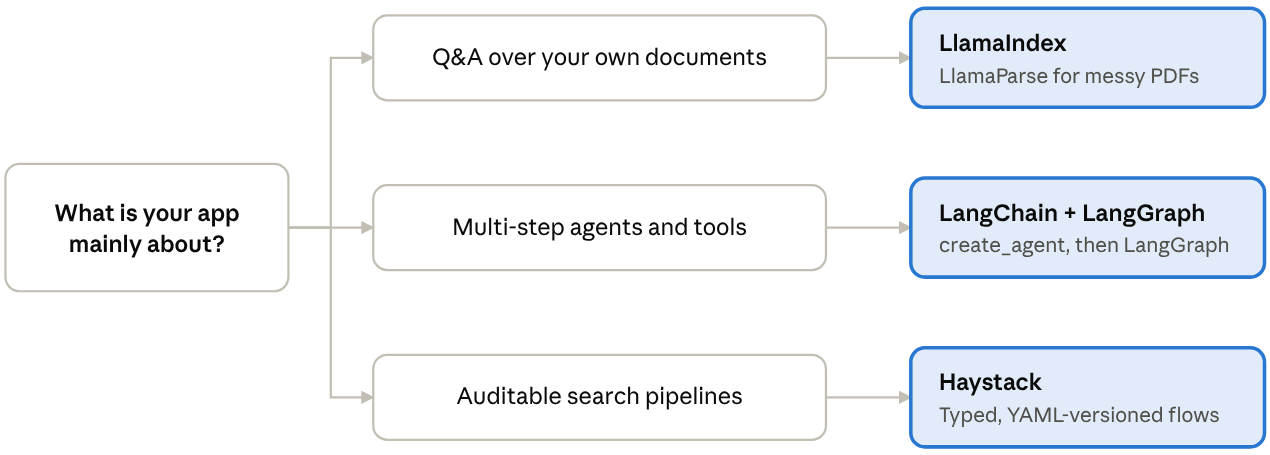
Dimension	LangChain (+ LangGraph)	LlamaIndex	Haystack
Learning curve	Moderate; steeper with LangGraph	Easiest for RAG; moderate for Workflows	Moderate; explicit but verbose
Best for	Complex agents, broad integrations, JS teams	Document-heavy RAG and knowledge assistants	Search-centric, auditable enterprise systems

6.2 The same task in each framework

Step	LangChain	LlamaIndex	Haystack
Load a PDF	`PyPDFLoader(path).load()`	`SimpleDirectoryReader(dir).load_data()`	`PyPDFToDocument().run(sources=[path])`
Chunk	`RecursiveCharacterTextSplitter`	`SentenceSplitter` (default)	`DocumentSplitter`
Index	`Chroma.from_documents(...)`	`VectorStoreIndex.from_documents(...)`	`DocumentWriter` into a Document Store
Retrieve	`store.as_retriever()`	`index.as_retriever()`	`InMemoryEmbeddingRetriever`
Answer	LCEL chain or `create_agent`	`index.as_query_engine().query(q)`	`ChatPromptBuilder` + `OpenAIChatGenerator` in a `Pipeline`
Add tools	`@tool` + `create_agent`	`FunctionTool` + `FunctionAgent`	`@tool` + `Agent`

7. Choosing a framework

Start from what your application is mainly about, then check the finer requirements in the table below. There is no wrong choice among the three for a standard RAG chatbot; the decision matters most at the edges.



Framework decision guide · three starting points

Each branch is a starting point, not a lock-in: all three frameworks can do all three jobs, and many teams combine them.

7.1 Recommendations by use case

Use case	First choice	Why	Good alternative
Chat with internal documents (policies, manuals)	LlamaIndex	Fastest path to good retrieval quality	Haystack
Complex PDFs: tables, scans, financial filings	LlamaIndex + LlamaParse	Best-in-class parsing of messy layouts	Haystack with a custom converter
Autonomous research or coding agent	LangChain + LangGraph	Mature agent loop, durable execution, middleware	LlamaIndex Workflows

Use case	First choice	Why	Good alternative
Multi-agent system with hand-offs and approvals	LangGraph	Explicit state graph, checkpoints, human-in-the-loop	LlamaIndex `AgentWorkflow`
Enterprise search with compliance or audit needs	Haystack	Typed, reviewable, YAML-versioned pipelines	LlamaIndex
Hybrid keyword plus semantic search	Haystack	BM25, dense retrievers, joiners and rankers built in	LlamaIndex `QueryFusionRetriever`
JavaScript / TypeScript web app	LangChain.js	Most complete JS port	LlamaIndex.TS
Quick prototype or hackathon	LlamaIndex or LangChain	Least code to a working demo	—
Structured data extraction from documents	LlamaIndex (LlamaExtract) or LangChain structured output	Schema-driven extraction	Haystack with a generator and JSON schema

7.2 Using them together

The frameworks are not mutually exclusive. Common combinations include:

- **LlamaIndex for retrieval inside a LangGraph agent**: build the index and query engine in LlamaIndex, wrap it as a tool, and orchestrate with LangGraph.
- **LlamaParse in front of any framework**: parse documents with LlamaParse, then hand the Markdown output to LangChain or Haystack.
- **Shared observability**: Langfuse, Arize Phoenix and other OpenTelemetry-based tools trace all three, so a mixed stack can still be monitored in one place.
- **Model Context Protocol (MCP)**: all three can call tools exposed by MCP servers, which makes tools portable across frameworks.

7.3 Best practices whichever you choose

1. **Pin versions.** All three move fast; pin exact versions in `requirements.txt` and upgrade deliberately.
2. **Get retrieval right before tuning prompts.** Most RAG failures are retrieval failures: check chunk size, overlap, metadata and whether the right passage was retrieved at all.
3. **Use hybrid search and a re-ranker** for production RAG; pure vector search misses exact terms such as product codes and names.
4. **Evaluate with a fixed test set.** Write 30 to 100 representative questions with expected answers and score faithfulness and context relevance on every change.
5. **Trace everything.** Turn on tracing from day one so you can see what was retrieved and sent to the model.
6. **Keep agents on a short leash.** Limit the tools, cap the number of iterations, and require human approval for actions with side effects.
7. **Stay model-agnostic.** Use each framework's provider-neutral model interface so you can switch between OpenAI, Anthropic, Google, Mistral or a local model.
8. **Protect data and secrets.** Keep API keys out of code and serialised configs, and filter personal data before it reaches external APIs.

8. Conclusion, glossary and references

The three frameworks have converged: each now offers RAG, tool calling, agents and observability hooks. What still separates them is their centre of gravity. LangChain, with LangGraph, is the agent and ecosystem leader; LlamaIndex is the document and retrieval specialist; Haystack is the explicit, production-pipeline framework. For a new project, prototype the core flow in the framework that matches your main need, measure retrieval and answer quality on a fixed test set, and only then optimise.

8.1 Glossary

Term	Meaning
Agent	An LLM in a loop that chooses tools and steps until a task is complete
BM25	A classic keyword-ranking algorithm used for sparse (lexical) search

Term	Meaning
Checkpointer	A LangGraph component that saves graph state after each step so runs can resume
Chunk / Node	A passage of a document stored and retrieved as one unit ("Node" in LlamaIndex)
Component	Haystack's unit of work: a class with typed inputs, outputs and a `run()` method
Embedding	A numeric vector representing the meaning of text
Hybrid search	Combining keyword (BM25) and vector search results
LCEL	LangChain Expression Language: pipe syntax for composing Runnables
MCP	Model Context Protocol: an open standard for exposing tools and data to LLM applications
Middleware	In LangChain 1.0, hooks that run around model and tool calls inside an agent
RAG	Retrieval-Augmented Generation: retrieving relevant data and adding it to the prompt
Re-ranker	A model that re-orders retrieved results by relevance to the query
Response synthesizer	A LlamaIndex component that turns retrieved Nodes into a final answer
Vector store / Document Store	A database that stores embeddings and supports similarity search
Workflow	In LlamaIndex, an event-driven set of async steps that orchestrate an application

8.2 Official resources

Framework	Documentation	Source code	Install
LangChain	docs.langchain.com	github.com/langchain-ai/langchain	`pip install langchain`
LangGraph	docs.langchain.com	github.com/langchain-ai/langgraph	`pip install langgraph`
LlamaIndex	docs.llamaindex.ai	github.com/run-llama/llama_index	`pip install llama-index`
Haystack	docs.haystack.deepset.ai	github.com/deepset-ai/haystack	`pip install haystack-ai`

8.3 Sources

- LangChain blog: LangChain and LangGraph 1.0 (October 2025)
- LangChain blog: LangChain and LangGraph 1.0 alpha releases (September 2025)
- LangChain changelog: LangChain 1.0 now generally available
- LlamaIndex blog: Announcing Workflows 1.0 (June 2025)
- Noze: Haystack, deepset's open-source RAG framework
- IDP Software: deepset vendor profile
- Jacar: Haystack: pipelines and agents

Code examples are illustrative and target the current major versions (LangChain 1.x, LlamaIndex with Workflows 1.x, Haystack 2.x); check each project's docs for exact signatures before use.